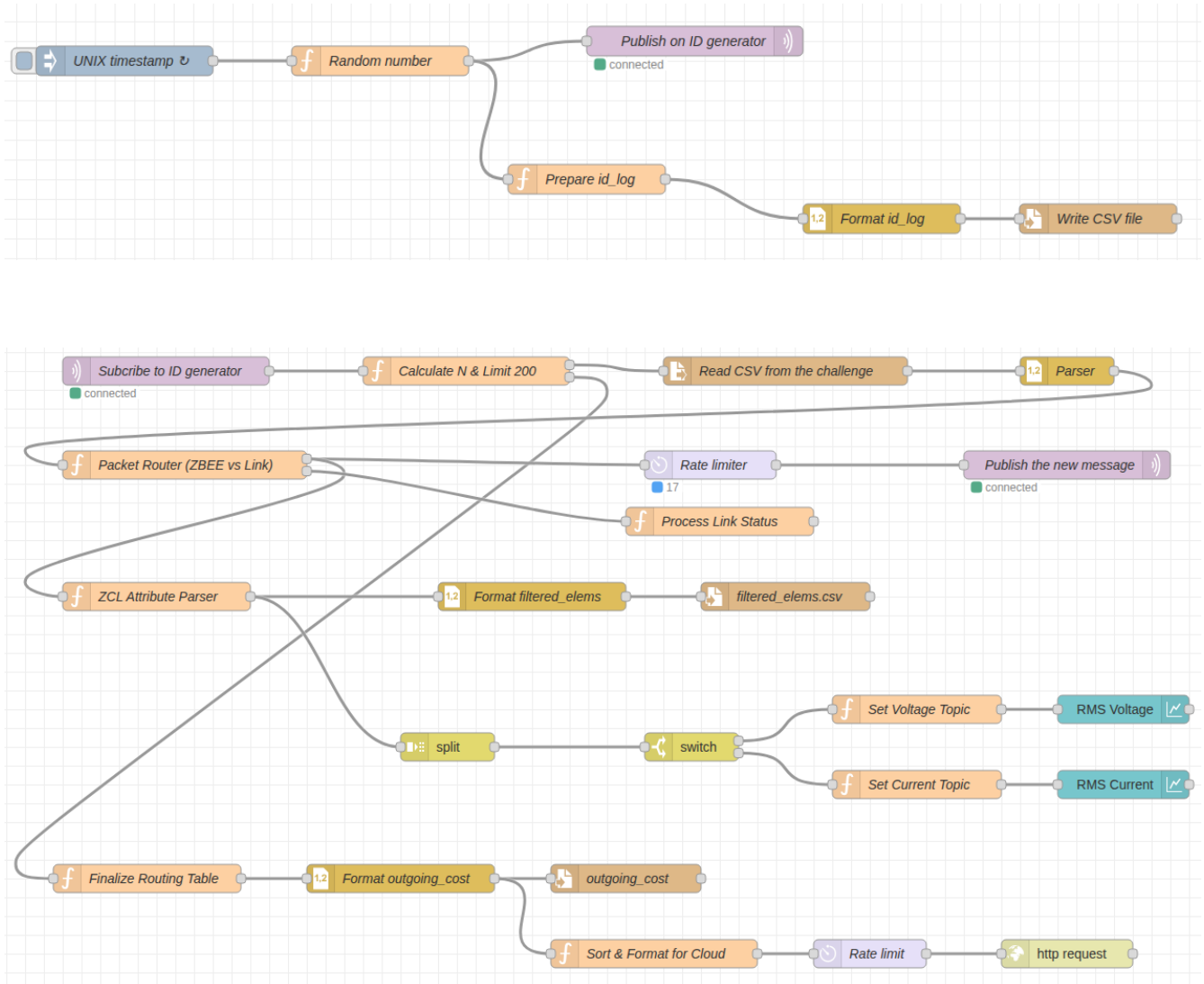


Challenge 3 IoT

Mattia Brianti 10773859
Alex Hathaway 10829946

1 Node-RED chart



2 Message Generation and Logging

2.1 UNIX timestamp

This inject node initiates the flow by triggering a timestamp payload at a fixed interval of 1 second. It acts as the heartbeat for the random ID generator, satisfying the requirement to periodically generate messages at a 1-second rate.

2.2 Random number

This `function` node uses standard JavaScript math libraries to generate a random ID between 0 and 30000. It extracts the current UNIX timestamp and formats the output into a JSON payload (`{"id": random_id, "timestamp": current_ts}`). Crucially, it manages a flow context variable (`genCount_finale`) to track the number of messages generated. Once the count reaches 200, the function prevents further execution, implicitly satisfying the strict limit of processing exactly 200 IDs.

2.3 Publish on ID generator

This `mqtt out` node receives the JSON payload from the random number generator and publishes it to the local Mosquitto broker (running on `localhost:1884`) under the topic `challenge3/id_generator`.

2.4 Prepare id_log

This `function` node intercepts the generated ID data to prepare it for CSV serialization. It restructures the payload to map precisely to the required CSV headers: `No.`, `ID`, and `TIMESTAMP`.

2.5 Format id_log & Write CSV file

The `Format id_log` is a standard `csv` formatter node configured to include column headers, outputting the correctly ordered string. Following it, the `Write CSV file` node appends the newly formatted string to `id_log.csv` in the local filesystem.

3 Message Ingestion and Modulo Processing

3.1 Subscribe to ID generator

This `mqtt in` node acts as the entry point for the second half of the flow. It listens to the `challenge3/id_generator` topic on the local broker, receiving the JSON payloads previously published.

3.2 Calculate N & Limit 200

This `function` node contains core logic. It reads the incoming JSON payload and evaluates $N = ID \bmod 5218$. It increments a local context counter, enforcing a strict hard stop after 200 messages are processed. To facilitate graceful completion, when the 200th message is evaluated, it simultaneously outputs an “END_ELABORATION” signal on its second output port, which triggers the finalization of the routing table logic later in the flow.

3.3 Read CSV from the challenge & Parser

Triggered by the calculation of N , the `Read CSV from the challenge` node loads the assignment’s `challenge3.csv` file. The `Parser csv` node transforms the raw text into structured JavaScript objects, preparing it for inspection.

4 Packet Routing and Logic Separation

4.1 Packet Router (ZBEE vs Link)

This `function` node filters the entire parsed CSV to find the specific row where the `Packet Number` equals the calculated N . It normalizes object keys (stripping extraneous whitespaces) and then examines the packet’s contents. If the `Command String` includes `Layer ZBEE_ZCL`, it dynamically builds a JSON payload with source/destination addresses and routes it to Output 1. If the `Packet Type` is `Link Status`, it extracts the relevant command string and source address, routing it to Output 2. All other packets are ignored, fulfilling the requirement to bypass non-relevant data.

5 ZCL Attribute Extraction and Publishing

5.1 Rate limiter & Publish the new message

For ZBEE_ZCL messages, a `delay` node configured as a rate limiter throttles throughput to a maximum of 10 messages per minute. The messages are then passed to the `Publish the new message mqtt out` node, which publishes the structured JSON payload to dynamically assigned topics (e.g., `ZigBee/ZB_SRC_DEV_NAME`).

5.2 ZCL Attribute Parser

This complex `function` node executes when a ZBEE packet contains “Active Power”, “RMS Current”, or “RMS Voltage”. It first cleans malformed single quotes from the command string to allow strict JSON parsing. It then iterates over the `Attribute` array, performing positional matching to associate each attribute with its corresponding `Status`, `Data Type`, and parsed value (extracted from the `Int16` or `Uint16` arrays).

Crucially, the node implements strict validation rules to ensure data integrity. It explicitly verifies the presence of the `Status`, `Data Type`, and `Data Value` for each individual attribute. If any of these fields are missing or undefined, the node actively excludes that attribute by skipping to the next iteration, rather than assigning default placeholder values (such as “0”). Attributes that pass this validation are then compiled into an array of objects matching the required CSV column structure.

5.3 Format filtered_elems & filtered_elems.csv

The array of parsed attributes is converted to CSV string format via the `csv` node, mapping to headers such as `Sequence Number`, `Attribute`, and `Data Value`. The `file` node then appends these rows to `filtered_elems.csv`.

6 Data Visualization

6.1 Split & Switch

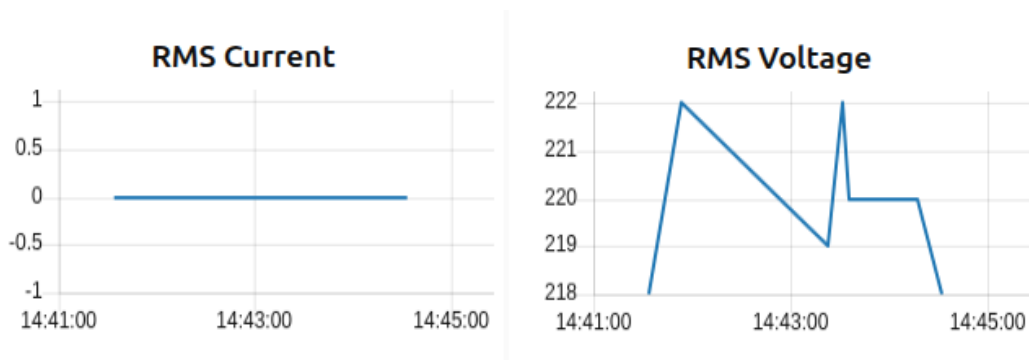
Because the Attribute Parser outputs an array of rows, a `split` node separates them into individual message objects. A `switch` node then inspects the `Attribute` property, routing “RMS Voltage” to Output 1 and “RMS Current” to Output 2.

6.2 Set Voltage Topic & Set Current Topic

These `function` nodes convert the raw string values into parsed floats to ensure the downstream charts render properly. They also convert the UNIX timestamps from seconds to milliseconds (a strict requirement for Node-RED `ui_chart` nodes) and set appropriate `msg.topic` labels.

6.3 RMS Voltage & RMS Current (UI Charts)

These `ui_chart` nodes plot the extracted real-time electrical metrics onto a Node-RED dashboard interface, visually satisfying the telemetry requirements.



7 Link Status and ThingSpeak Cloud Integration

7.1 Process Link Status

When a “Link Status” packet is detected, this `function` node parses the nested `Links` array within the command string. It extracts the outgoing costs for each destination address and stores/updates this information in a globally scoped dictionary (`global.get("routingTable")`), ensuring that the latest costs are continuously maintained throughout the run.

7.2 Finalize Routing Table

Triggered exclusively by the “END_ELABORATION” signal sent by the modulo calculator on the 200th iteration, this `function` node accesses the global routing table. It flattens the nested dictionary into an array of flat objects containing `Source`, `Destination`, and `Cost`.

7.3 Format outgoing_cost & outgoing_cost.csv

The flattened array is converted to CSV text and written to the `outgoing_cost.csv` file. The node is explicitly configured to overwrite the file (`overwriteFile: "true"`), providing a clean snapshot of the final link states.

7.4 Sort & Format for Cloud

Following the creation of the final routing table, this `function` node fulfills the ThingSpeak integration requirement. It extracts all source addresses, sorts them mathematically by treating them as hexadecimal integers, and isolates the single smallest source address. It then extracts all destinations associated with that source, sorts them in ascending hexadecimal order, and creates an array of messages where the payload contains the routing cost.

7.5 Rate limit & HTTP Request

To adhere to ThingSpeak’s API rate limits, a `delay` node queues the sorted cost messages and releases them at a strict rate of 1 message every 20 seconds. Finally, the `http request` node performs a GET request to the ThingSpeak Update API, appending the routing cost dynamically into `field1` via mustache templating.

